

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1701

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

1. Analytical Problem Solving

Code of Conduct:

*I **_SHAIK THOUHID (192421280)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

thouhid

CO1-AT1-CSA1701

NAME : SHAIK THOUHID

REGNO:192421280

Annexure A: Analytical Problem Solving

1 Water Jug Problem Solution

Problem Statement

You are given 2 jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug?

Explicit Assumptions:

- A jug can be filled from the pump.
- Water can be poured out of a jug onto the ground.
- Water can be poured from one jug to another.
- There are no other measuring devices available.

Formal State-Space Formulation

To solve the Water Jug problem using Artificial Intelligence state-space search techniques, we define the components as follows:

1. **State Space (XS):** A state is represented as an ordered pair (x, y) , where:
 - x represents the volume of water in the 4-gallon jug ($0 \leq x \leq 4$).
 - y represents the volume of water in the 3-gallon jug ($0 \leq y \leq 3$).
2. **Initial State (S_0):** $(0, 0)$ — Both jugs are initially empty.
3. **Goal State (G):** $(2, y)$ for any $0 \leq y \leq 3$ (specifically, exactly 2 gallons in the 4-gallon jug).
4. **Production Rules / Operators:** The following set of operations can be applied to transition from one state (x, y) to another:

- **Fill 4-gallon jug:** $\text{If } x < 4 \text{, then } (4, y)$
- **Fill 3-gallon jug:** $\text{If } y < 3 \text{, then } (x, 3)$
- **Empty 4-gallon jug onto the ground:** $\text{If } x > 0 \text{, then } (0, y)$
- **Empty 3-gallon jug onto the ground:** $\text{If } y > 0 \text{, then } (x, 0)$ -
Pour water from 3-gallon jug into 4-gallon jug until 4-gallon jug is full: $\text{If } x + y \geq 4 \text{ and } y > 0 \text{, then } (4, y - (4 - x))$
- **Pour water from 4-gallon jug into 3-gallon jug until 3-gallon jug is full:**
 $\text{If } x + y \geq 3 \text{ and } x > 0 \text{, then } (x - (3 - y), 3)$
- **Pour all water from 3-gallon jug into 4-gallon jug (sum ≤ 4):** $\text{If } x + y \leq 4 \text{ and } y > 0 \text{, then } (x + y, 0)$
- **Pour all water from 4-gallon jug into 3-gallon jug (sum ≤ 3):** $\text{If } x + y \leq 3 \text{ and } x > 0 \text{, then } (0, x + y)$

Step-by-Step Solution Sequence

There are multiple valid sequences of states to reach the goal. Below is the most direct sequence:

1. **Initial State:** $(0, 0)$ — Both jugs are empty.
2. **Fill the 3-gallon jug:** $(0, 3)$ using rule (Fill 3-gallon jug).
3. **Pour water from 3-gallon jug into 4-gallon jug:** $(3, 0)$ using rule (Pour all water from 3-gallon into 4-gallon).
4. **Fill the 3-gallon jug again:** $(3, 3)$ using rule (Fill 3-gallon jug).
5. **Pour water from 3-gallon jug into 4-gallon jug until 4-gallon jug is full:** $(4, 2)$ using rule (Pour from 3-gallon to 4-gallon until full; since $3+3=6 \geq 4$, 4-gallon gets filled to 4, and $3 - (4-3) = 2$ remains in the 3-gallon jug).
6. **Empty the 4-gallon jug onto the ground:** $(0, 2)$ using rule (Empty 4-gallon jug).
7. **Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug:** $(2, 0)$ using rule (Pour all water from 3-gallon into 4-gallon).

Goal Reached: The 4-gallon jug now contains exactly 2 gallons of water: $(2, 0)$.

2. Mars Rover Analysis and Exploration

Problem Statement

Find out how Mars Rover analyses and explores the samples on Mars by answering the following five core AI agent questions.

i. What are the percepts for this agent?

The percepts represent the inputs that the Mars Rover receives from its physical environment at any given instant through its sensory apparatus. For a Mars Rover, percepts include:

- **Visual Data:** High-resolution stereoscopic imagery, panoramic landscape photos, and microscopic camera shots of rocks and soil captured by navigation and hazard cameras (Navcams, Hazcams, Mastcam).
- **Spectroscopic Readings:** Data from instruments like ChemCam or SuperCam, capturing laser-induced breakdown spectroscopy (LIBS) to determine elemental composition.
- **X-Ray and Mineralogical Data:** Readings from spectrometers (such as PIXL or SHERLOC) identifying organic compounds and mineral structures.
- **Kinetic & Inertial Feedback:** Wheel slip measurements, tilt angles, motor current loads, acceleration, and orientation from IMUs (Inertial Measurement Units).
- **Environmental Conditions:** Ambient temperature, atmospheric pressure, wind speed, radiation levels, and weather data from MEDA (Mars Environmental Dynamics Analyzer).
- **Command Inputs:** Direct or scheduled telecommunication signals and software instructions transmitted from mission control operators on Earth.

ii. Characterize the operating environment.

Using standard AI environment classification criteria, the Mars Rover's operating environment can be characterized as follows:

- **Partially Observable:** The rover cannot simultaneously sense the entire planet or hidden subsurface geology. Its sensors are limited by line-of-sight, terrain occlusion, dust storms, and buried formations.
- **Stochastic (Non-Deterministic):** Actions do not have completely predictable outcomes. Wheel slippage in loose Martian sand, unexpected rock fracturing, sudden dust devils, and communication latency introduce environmental uncertainty.
- **Sequential:** Current actions (e.g., choosing a path around a crater) directly impact future states and long-term mission success. Decisions are part of a multi-step sequence.
- **Dynamic:** The Martian environment changes independently of the agent (e.g., wind blowing dust, weather changes, shifting dunes). However, relative to rapid processing, parts of the local rock terrain can be treated as semi-static during immediate pathfinding.
- **Continuous:** Time, spatial coordinates, steering angles, power levels, and arm articulation operate over continuous mathematical domains rather than discrete states.
- **Multi-Agent:** While primarily a single agent on the surface, it interacts with orbiting relay satellites (e.g., MRO, MAVEN), Earth-based control teams, and potentially other robotic assets, making it part of a complex cooperative multi-agent framework.

iii. What are the actions the agent can take?

The effector and actuator systems of the Mars Rover enable it to execute various physical and computational actions:

- **Locomotion:** Steering, driving forward or backward, turning in place, and traversing slopes or navigating around obstacles (rocks, sand ripples).
- **Arm Articulation & Positioning:** Deploying and positioning the robotic arm to reach specific rock faces or soil patches.
- **Sampling & Coring:** Drilling into rocks using a coring drill, extracting rock core samples, sealing sample tubes, and caching them.
- **Scientific Analysis:** Firing lasers for spectroscopy, operating microscopic cameras, activating ultraviolet lasers to scan for organic biomarkers, and logging atmospheric data.
- **Communication:** Pointing high-gain antennas towards Earth or orbiting satellites, transmitting compressed data packages, and receiving software patches or daily action sequences.
- **Power & Thermal Management:** Adjusting solar panel angles (if applicable), turning on internal survival heaters to protect electronics against extreme Martian night cold, and entering low-power sleep modes.

iv. How can one evaluate the performance of the agent?

The Performance Measure (utility function) for a Mars Rover is defined by mission planners and scientists based on the following key metrics:

- **Scientific Yield:** Quantity, diversity, and high scientific value of collected data, rock samples, and geochemical discoveries returned or analyzed.
- **Safety and Integrity:** Freedom from catastrophic structural damage, tipping, getting permanently stuck in sand traps, or sustaining critical hardware failures.
- **Mission Efficiency:** Energy optimization, efficient battery utilization, data transmission throughput, and distance successfully traversed per sol (Martian solar day).
- **Autonomy and Reliability:** Ability to successfully execute autonomous navigation (e.g., AutoNav software) without requiring constant human intervention or correction for communication delays.
- **Goal Completion:** Successful completion of designated milestone tasks within scheduled mission timelines.

v. What sort of agent architecture do you think is most suitable for this agent? Why?

- **Selected Architecture:** A Hybrid (Layered / Hierarchical) Agent Architecture combining Model-Based Reflex Agents with Goal-Based and Utility-Based Planning Layers .

- **Justification:**
 - Mars operates under significant communication delays (ranging from 4 to 24 minutes one-way), making real-time teleoperation impossible for immediate hazards.
 - The **Reactive/Reflex Layer** handles immediate local safety (e.g., hazard detection stopping the wheels instantly if a dangerous cliff or tilt is sensed).
 - The **Deliberative/Goal-Based Layer** handles complex daily path planning, science target selection, and sequence optimization based on internal world models and scientific utility.
 - This hybrid separation ensures immediate physical survival while enabling sophisticated long-term scientific exploration.
-

3. 8-Queens Problem Formulation and Search Strategies

Problem Statement

Explore the search space using search strategies and formulate the problem components for the 8-Queens problem. Place 8 queens on a chessboard such that none of the queens attack any of the others. A configuration of 8 queens on the board as shown in traditional layouts places them such that no two queens share the same row, column, or diagonal.

Problem Components Formulation

1. Initial State:

- Can vary depending on the formulation strategy.
- *Formulation A (Incremental formulation):* An empty chessboard with zero queens placed (0×0 board or empty board).
- *Formulation B (Complete-state formulation):* All 8 queens placed on the board, typically one per column or row, allowing multiple queens to potentially attack each other initially.

2. State Space:

- *For incremental formulation:* All possible arrangements of k queens ($0 \leq k \leq 8$) on the 8×8 board. The search tree has branching factor $b = 64$ at depth 1, etc.
- *For complete-state formulation:* $8^8 = 16,777,216$ possible board configurations where each queen is placed in a column.

3. Successor Function / Operators:

- *Incremental*: Add a queen to any empty square on the board.
- *Complete-state*: Move a queen within its current column to any other square in that same column.

4. Goal Test:

- Check if all 8 queens are placed on the board and the number of attacking pairs is **zero** (i.e., no two queens share the same row, column, or diagonal).

5. Path Cost / Objective Function:

- For uninformed search, cost is uniform per step.
- For heuristic/local search, the objective function $h(x)$ counts the number of attacking pairs of queens on the board. The goal is to find a state where $h(x) = 0$.

Exploration Using Search Strategies

1. Uninformed Search (Depth-First Search / Breadth-First Search):

- Naive BFS on the full state space of placing 8 queens anywhere results in a search tree depth of 8 with branching factor 64, yielding $64^8 \approx 2.8 \times 10^{14}$ states, which is computationally expensive.
- Restricting placement to exactly one queen per column reduces the state space to $8^8 = 16,777,216$ states, which can be searched more efficiently using backtracking depth-first search.

2. Informed & Heuristic Search (Hill Climbing & Simulated Annealing):

- **Steepest-Ascent Hill Climbing**: Start with a random configuration of 8 queens (one per column). Evaluate all possible single-column moves for all queens, and move to the configuration that reduces the heuristic $h(x)$ (number of conflicts) the most.
- *Limitation*: Prone to local minima, plateaus, and shoulders where neighboring states do not decrease $h(x)$.
- **Simulated Annealing / Genetic Algorithms**: To overcome local minima in the 8-Queens problem, stochastic local search algorithms like simulated annealing accept worse moves with a decaying probability, successfully finding conflict-free configurations ($h(x) = 0$) almost instantly.

4. OLA Cab Booking Problem-Solving Agent & Pseudocode

Problem Statement

A customer wants to travel from one location to another location using the OLA Cab booking mobile application. The customer can select any of the cab types as mini, micro, sedan, shared, prime, and so on based on comfort and budget. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Identification of Problem-Solving Agent Type

- **Agent Type:** **Goal-Based Agent** combined with a **Utility-Based Agent** .
- **Justification:**
- The agent has a clear explicit goal: transport the user from pickup location \$A\$ to destination location \$B\$.
- It functions as a **Utility-Based Agent** because among multiple valid paths and cab types (mini, micro, sedan, shared, prime), it evaluates trade-offs between cost, estimated time of arrival (ETA), vehicle comfort, and surge pricing to maximize user satisfaction (utility).

Pseudocode for OLA Cab Booking Agent

// Pseudocode for OLA Cab Booking Problem-Solving Agent Algorithm

OLA_Cab_Booking_System:

Input: pickup_location, destination_location, user_preferences (comfort, max_budget)

Output: Confirmed ride details, assigned driver, and route plan

// Step 1: Initialize Problem Environment and State

current_state ← GetGPSLocation(user) goal_state ←

destination_location

```

// Step 2: Discover Available Cab Types & Drivers in Vicinity available_cabs ←
ScanNearbyDrivers(pickup_location, radius = 5km)

if available_cabs is Empty then

    Return "No cabs available nearby. Please try again later."

end if

// Step 3: Filter and Categorize Cab Options valid_options ←
[]

For each cab in available_cabs do cab_category ← cab.type // e.g., Mini, Micro, Sedan, Prime,
Shared estimated_fare ← CalculateFare(cab_category, pickup_location, destination_location)

eta ← EstimateArrivalTime(cab.location, pickup_location)

// Evaluate against user preferences (Utility function) if
estimated_fare <= user_preferences.max_budget then

    utility_score ← EvaluateUtility(cab_category, estimated_fare, eta,
user_preferences.comfort)

    Append (cab, cab_category, estimated_fare, eta, utility_score) to valid_options
end if End For if

valid_options is Empty then

    Return "No cabs match your budget constraints."

end if

```

```

// Step 4: Select Optimal Cab based on Maximum Utility

selected_option ← SelectMaxUtility(valid_options) // Step 5:

Execute Booking Transaction & Goal Formulation

booking_status ← ConfirmBookingWithDriver(selected_option.cab, pickup_location,
destination_location)

if booking_status == SUCCESS then

    // Step 6: Route Planning and Navigation (Problem Solving Search) route_path ←
    ComputeOptimalRoute(pickup_location, destination_location, TrafficData)

    StartNavigation(selected_option.driver, route_path)

    Return Details(selected_option, route_path, estimated_time_of_arrival)

else

    Return "Booking failed. Driver cancelled or unavailable. Retrying..."

end if

End Algorithm

```

5. Logistics Company Uniform Cost Search (UCS)

Problem Statement

A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost ($\text{fuel} + \text{time}$ \$). The company wants to determine the least-cost path from the starting warehouse $\mathbf{S}$ to the destination warehouse $\mathbf{G}$ using the **Uniform Cost Search (UCS)** algorithm.

The delivery network is represented as a weighted graph with nodes $\mathbf{S}$,

$\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$, and $\mathbf{G}$, and edge weights representing travel costs as shown in the diagram:

- Edge $(S \rightarrow A)$ with cost 1
- Edge $(S \rightarrow C)$ with cost 1
- Edge $(S \rightarrow G)$ with cost 12
- Edge $(A \rightarrow B)$ with cost 3
- Edge $(A \rightarrow C)$ with cost 1
- Edge $(B \rightarrow D)$ with cost 3
- Edge $(C \rightarrow D)$ with cost 1
- Edge $(C \rightarrow G)$ with cost 2
- Edge $(D \rightarrow G)$ with cost 3

(Note: Edge values interpreted from standard weighted graph structures matching the diagram layout).

Uniform Cost Search (UCS) Algorithm Overview

Uniform Cost Search expands the node n with the lowest path cost $g(n)$ in the frontier queue. It is complete and optimal when edge costs are non-negative.

Step-by-Step Execution Trace:

1. Initialization:

- **Frontier (Priority Queue ordered by cumulative path cost g):** - $[(\mathbf{S}, g=0)]$
- **Explored Set:** $\emptyset$

2. Iteration 1:

- Pop node with lowest cost from frontier: $\mathbf{S}$ with $g(\mathbf{S}) = 0$.
- Is $\mathbf{S}$ the goal $\mathbf{G}$? No.
- Add $\mathbf{S}$ to Explored Set. Explored = $\{\mathbf{S}\}$.
- Expand successors of $\mathbf{S}$:
 - Edge $S \rightarrow A$ (cost 1): Path $S \rightarrow A$, cumulative cost $g = 0 + 1 = 1$.
 - Edge $S \rightarrow C$ (cost 1): Path $S \rightarrow C$, cumulative cost $g = 0 + 1 = 1$.
 - Edge $S \rightarrow G$ (cost 12): Path $S \rightarrow G$, cumulative cost $g = 0 + 12 = 12$.
- **Frontier:** $[(A, 1), (C, 1), (G, 12)]$

3. Iteration 2:

- Pop node with lowest cost: A with $g(A) = 1$.
- Is A the goal $\mathbf{G}$? No.
- Add A to Explored Set. Explored = $\{\mathbf{S}, A\}$.
- Expand successors of A :
 - Edge $A \rightarrow B$ (cost 3): Path $S \rightarrow A \rightarrow B$, cumulative cost $g = 1 + 3 = 4$.
 - Edge $A \rightarrow C$ (cost 1): Path $S \rightarrow A \rightarrow C$, cumulative cost $g = 1 + 1 = 2$. (Already in frontier via $S \rightarrow C$ with cost 1; keep lower cost 1).
- **Frontier:** $[(C, 1), (B, 4), (G, 12)]$

4. Iteration 3:

- Pop node with lowest cost: C with $g(C) = 1$ (path $S \rightarrow C$).
- Is C the goal $\mathbf{G}$? No.
- Add C to Explored Set. Explored = $\{\mathbf{S}, A, C\}$.
- Expand successors of C :
 - Edge $C \rightarrow D$ (cost 1): Path $S \rightarrow C \rightarrow D$, cumulative cost $g = 1 + 1 = 2$.
 - Edge $C \rightarrow G$ (cost 2): Path $S \rightarrow C \rightarrow G$, cumulative cost $g = 1 + 2 = 3$.
- **Frontier (sorted by cost):** $[(D, 2), (G, 3), (B, 4), (G, 12)]$

5. Iteration 4:

- Pop node with lowest cost: D with $g(D) = 2$ (path $S \rightarrow C \rightarrow D$).
- Is D the goal $\mathbf{G}$? No.
- Add D to Explored Set. Explored = $\{\mathbf{S}, A, C, D\}$.
- Expand successors of D :
 - Edge $D \rightarrow G$ (cost 3): Path $S \rightarrow C \rightarrow D \rightarrow G$, cumulative cost $g = 2 + 3 = 5$.
- **Frontier (sorted by cost):** $[(G, 3), (B, 4), (G, 5), (G, 12)]$

6. Iteration 5:

- Pop node with lowest cost: G with $g(G) = 3$ (path $S \rightarrow C \rightarrow G$).
- Is G the goal? **YES!**

Final Result & Optimal Path

- **Optimal Least-Cost Path:** $S \rightarrow C \rightarrow G$ -
Total Minimum Cost: $1 + 2 = 3$ units.